Research Glossary

Terms from the Lunar Lander reward-shaping project, explained in plain words, with examples from our own runs. Compiled 9 November 2026 for the research paper.

1. Reinforcement learning basics

Reinforcement learning (RL)

A way of training an AI by trial and error. It tries actions, gets points (rewards) for good results and loses points for bad ones, and slowly learns which actions earn the most.

> *In our project:* The rocket learned to land with no one telling it how. It only saw its score.

Agent

The AI that makes the decisions.

> *In our project:* The neural network that chooses which engine to fire each frame.

Environment

`In the code: env = gym.make("LunarLander-v3")`

The world the agent lives in. It runs the physics and hands out rewards.

> *In our project:* LunarLander-v3, the moon-landing game from the Gymnasium library.

Observation (state)

`In the code: obs, obs[0] to obs[7]`

The information the environment gives the agent each frame. The agent never sees the picture, only these numbers.

> *In our project:* 8 numbers: left/right position, height, sideways speed, up/down speed, tilt, spin, left leg touching, right leg touching.

Action and action space

`In the code: env.action_space`

An action is one choice the agent makes. The action space is the full set of allowed choices. Discrete means a fixed list of separate options, like buttons.

> *In our project:* Discrete(4): 0 = do nothing, 1 = left side engine, 2 = main engine, 3 = right side engine. We found that engine 1 pushes left and spins the rocket anticlockwise, and engine 3 pushes right and spins it clockwise.

Reward

The points the agent gets for one frame. It can be positive or negative.

> *In our project:* Firing the main engine costs 0.3 points per frame and a side engine costs 0.03. Landing gives +100 and crashing gives −100.

Step (frame)

`In the code: env.step(action)`

One tick of the game. The agent picks an action, the physics moves forward by 1/50th of a second, and the environment returns the new observation and the reward.

Episode (attempt)

One full try, from the start position until the rocket lands, crashes or runs out of time.

> *In our project:* A random agent's episode lasted about 60 to 100 frames. Our best agent landed in about 300.

Reset

`In the code: env.reset(seed=0)`

Starts a new episode. It puts the rocket back at the top with a random push. It does not set the numbers to zero.

Seed
A number that fixes the randomness, so the same seed always gives the same starting push. This makes experiments repeatable and comparisons fair.

In our project: We tested on seeds 100 to 119, so every agent faced the same 20 starts.

Terminated and truncated
Two ways an episode ends. Terminated means it ended because of what happened (landing or crash). Truncated means the clock ran out at 1,000 frames.

In our project: Truncated episodes were our sign of stalling and hacking.

Return (score)
All the rewards in one episode added together. A return of 200 or more counts as solving Lunar Lander.

In our project: Random agent: about −206 on average. Best trained agent: about 270.

2. How the game works

Built-in reward (shaping inside the game)
Each frame, the game pays for getting closer to the pad, slower and more upright, charges for fuel, and gives +10 for each leg that touches down.

At rest (asleep)
The game only counts a landing, and gives the +100, once the rocket is completely still with no engines firing for a short time. Any engine pulse wakes it up again.

In our project: This rule is what made the ground-stall hack possible.

Time limit
Every episode is cut off at 1,000 frames (20 seconds of game time) if it has not ended by itself.

3. Training

Neural network
`In the code: "MlpPolicy" (MLP = multi-layer perceptron)`

A function made of layers of numbers that turns inputs into outputs. Training adjusts those numbers.

In our project: Ours takes the 8 observation numbers in and gives a chance for each of the 4 actions.

Policy
The agent's rule for choosing actions. In our case the policy is the neural network itself.

PPO (Proximal Policy Optimization)
`In the code: PPO("MlpPolicy", env, ...)`

The training algorithm we used. It lets the agent play, then makes actions that turned out better than expected a little more likely, and worse ones a little less likely. "Proximal" means it limits how far the policy can change in one update, so training stays stable.

Actor and critic
PPO trains two networks. The actor picks actions. The critic predicts how good a situation is (the score likely to follow). The actor uses the critic's predictions to judge whether an action did better or worse than expected.

Timesteps
`In the code: total_timesteps=1_000_000`

The total number of frames used for training.

In our project: Each of our training runs used 1,000,000 timesteps and took about 21 to 24 minutes on Colab.

Vectorised environments

`In the code: make_vec_env(..., n_envs=16)`

Several copies of the game run side by side, so the agent gathers experience faster.

Hyperparameters

Settings chosen before training, not learned. Ours: n_steps = 1024 (frames per copy before each update), batch_size = 64, n_epochs = 4, gamma = 0.999, gae_lambda = 0.98, ent_coef = 0.01, learning rate = 0.0003.

Gamma (discount factor)

How much the agent cares about future rewards. Close to 1 means rewards far in the future still matter a lot.

> *In our project:* gamma = 0.999, because the landing reward comes only at the end.

Exploration and entropy

`In the code: ent_coef=0.01, entropy_loss`

Exploration means trying actions the agent is not sure about. Entropy measures how random the policy still is. The entropy coefficient gives a small bonus for staying a bit random, so the agent keeps exploring.

> *In our project:* entropy_loss started at −1.38 (fully random over 4 actions) and moved toward 0 as the agent grew confident.

ep_rew_mean and ep_len_mean

The average score and average length of recent episodes during training. In our setup the score recorder sits inside our wrappers, so ep_rew_mean shows the real game score without our added bonuses or fees.

explained_variance and value_loss

How well the critic predicts scores. explained_variance near 1 is good and near 0 means it knows nothing. value_loss is the critic's error, so lower is better.

> *In our project:* In Run 1, explained_variance rose from 0 to 0.98.

Local optimum

A strategy that is good enough that the agent stops searching, even though a better one exists.

> *In our project:* With dice off, the Run 3 agent landed normally instead of farming, partly because it learned landing first.

Saved model

`In the code: model.save(...), PPO.load(...)`

The trained network's numbers stored in a file so they can be loaded later without retraining.

> *In our project:* ppo_lander.zip, ppo_timefee.zip, ppo_hover.zip, ppo_fixed.zip

4. Testing (evaluation)

Baseline

The score of a simple reference agent, used to show how much training improved things.

> *In our project:* A random agent averaged −206.2 over 10 episodes.

Deterministic evaluation ("dice off")

`In the code: model.predict(obs, deterministic=True)`

Testing where the agent always presses the action it rates most likely. This is the usual way agents are tested.

> *In our project:* Run 3 looked perfect this way: 0 of 20 episodes hit the time limit.

Stochastic evaluation ("dice on")

`In the code: model.predict(obs, deterministic=False)`

Testing where the agent rolls a weighted die over its action chances, the same way it behaved during training.

> *In our project:* Run 3 hit the time limit in 16 of 20 episodes this way. The hack only showed up with dice on.

Held-out test seeds

Test starts the agent was not checked on before, so the test is fair.

> *In our project:* Seeds 100 to 119.

Training log versus evaluation

The training log shows behaviour while exploring (dice on). A separate test shows behaviour under fixed conditions. They can disagree a lot.

> *In our project:* Run 2's training log said 833 frames per episode. The dice-off test said 311.

Run-to-run variation

Training the same setup again with a different random seed can give a noticeably different agent. Strong claims need several training runs per condition, reported as a mean and a spread.

> *In our project:* The pilot trained each version once. The main study will train each condition 5 times.

5. Rewards and reward hacking

Reward function

The rule that decides how many points each frame is worth. Designing it is called reward design.

Reward shaping

Adding extra rewards to guide learning, on top of the main goal's reward.

> *In our project:* The time fee, the hover bonus and the improvement bonus are all reward shaping.

Wrapper

`In the code: class TimePenalty(gym.Wrapper)`

A layer placed around the environment that changes one thing (such as the reward) and passes everything else through unchanged. Wrappers can be stacked.

> *In our project:* RecordVideo, TimePenalty, HoverBonus and ImprovementBonus are all wrappers.

Time penalty (fee)

`In the code: TimePenalty(env, fee=0.05)`

A small cost charged every frame, so dawdling is expensive.

> *In our project:* A fee of 0.05 per frame made the agent land faster (311 frames against 478) and stopped the ground stall in our tests.

State-based bonus

`In the code: HoverBonus(env, bonus=1.0)`

A reward for being in a good state. It is paid again on every frame the agent stays there, so it can be farmed.

> *In our project:* The hover bonus: +1 every frame the rocket was upright and slow (tilt, sideways speed and up/down speed all under 0.1).

Reward hacking (specification gaming)

When an agent earns a high reward in a way the designer did not intend, by exploiting a gap between what the reward measures and what the designer actually wanted.

> *In our project:* Run 3 landed, then kept pulsing both side engines so it never counted as at rest, collecting the hover bonus on 795 of 1,000 frames (799 frames with both legs down).

Proxy reward
A measurable stand-in for what you actually want. Hacking happens when the agent optimises the proxy instead of the real goal.

> *In our project:* "Upright and slow" was a proxy for "land gently".

Ground stall
Our name for the hack where the rocket lands but keeps firing small pulses, so the episode never ends.

> *In our project:* It also appeared once under the default reward (Run 1, seed 105): touchdown at frame 361, then 346 frames of no action and 293 frames of engine 3, ending at the time limit with a score of 120.6.

Potential function
`In the code: potential(obs) = −(|obs[4]| + |obs[2]| + |obs[3]|)`

A score for how good a state is. We used minus the sum of the absolute tilt, sideways speed and up/down speed, so 0 is best.

Potential-based reward shaping
`In the code: ImprovementBonus(env, scale=10.0)`

Paying for improvement instead of for state: reward = scale × (new potential − old potential). Introduced by Ng, Harada and Russell (1999), who proved it does not change which strategy is best.

> *In our project:* Run 4 used scale = 10. It never stalled, with dice off or on, and scored highest (270.6).

Telescoping sum
When per-frame differences are added up, everything in the middle cancels, leaving only end minus start. Like a staircase: your total is top step minus bottom step, however you wander.

> *In our project:* Our check: total improvement bonus 11.1195 equalled 10 × (end − start) = 11.1195. Sitting still therefore earns nothing.

Note on gamma in potential-based shaping
The textbook formula multiplies the new potential by gamma (gamma × new − old). We used new − old. With gamma = 0.999 the difference is tiny, but the paper should state which version was used.

6. Research terms

Hypothesis
A testable prediction written before collecting data. It names what you change and what you compare. Journals often ask for the form "We hypothesized that…".

Pilot study
A small exploratory experiment that shows something worth testing properly.

> *In our project:* Our Runs 1 to 4: one training run each, 20 test episodes.

Main study
The larger, planned experiment that tests the hypothesis with repeats and fixed rules.

> *In our project:* Planned: hover bonus of 0, 0.25, 0.5, 1 and 2 plus the improvement bonus, 5 training runs each, 100 test episodes in each mode.

Condition and control
A condition is one version of the setup being tested. The control is the version without the change, used for comparison.

> *In our project:* Control: fee only. Conditions: fee plus each bonus size.

Independent and dependent variables

The independent variable is what you change on purpose. The dependent variable is what you measure.

> *In our project:* Independent: bonus size and evaluation mode. Dependent: hack rate, score and episode length.

Hack rate

The share of test episodes where hacking happened. Its exact definition must be fixed before the main study.

> *In our project:* Draft: an episode is hacked if it reaches the 1,000-frame limit with both legs on the ground.

Related work and contribution

Related work is the earlier research your paper builds on. The contribution is what your paper adds that was not known before.

> *In our project:* Closest prior work: Jang et al. (2025) found deterministic evaluation hides weaknesses, but did not study reward hacking. That gap is our contribution.

Preprint and endorsement

A preprint is a paper shared publicly before peer review, for example on arXiv. First-time arXiv authors need an endorsement from an existing author.

AI-assistance disclosure

A statement, usually in the acknowledgments, saying how AI tools were used in the work.

7. Pilot results at a glance

Run	Reward	Dice OFF score / length / time limits	Dice ON score / length / time limits
Random	none	−206.2 average (10 episodes)	–
1	Default game reward	235.2 / 478 / 1 of 20	not measured
2	Fee 0.05	264.3 / 311 / 0 of 20	256.5 / 460 / 1 of 20
3	Fee + hover bonus 1.0	265.0 / 312 / 0 of 20	134.5 / 838 / 16 of 20
4	Fee + improvement bonus ×10	270.6 / 293 / 0 of 20	269.6 / 273 / 0 of 20

Test seeds 100 to 119 for Runs 1 to 4. Each run was trained once. Dice-on tests were not seeded, so repeating them gives slightly different numbers (Run 3 gave 17 of 20 the first time).